

01A JS Intro and data types

JavaScript Introduction and Data Types: Complete Lab Guide

Prerequisites: VS Code with Live Server extension!

Setup Instructions

1. **Create your workspace:**

- Create a new folder on your desktop called `js-basics`
- Open this folder in VS Code (File → Open Folder)

2. **Create your files:**

- Create a file called `index.html`
- Create a file called `script.js`

3. **Set up your HTML file:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript Basics</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 800px;
      margin: 50px auto;
      padding: 20px;
      background-color: #f5f5f5;
    }
    h1 {
      color: #333;
    }
  </style>
</head>
<body>
  <h1>JavaScript Console Lab</h1>
  <p>Open the browser console (F12 or Right-click → Inspect → Console) to see the
  output!</p>

  <script src="script.js"></script>
</body>
</html>
```

4. Launch Live Server:

- Right-click on `index.html` and select "Open with Live Server"
- Your browser will open automatically
- Press **F12** to open Developer Tools and click on the **Console** tab

Part 1: What is JavaScript?

JavaScript is a programming language that makes web pages interactive. Think of it as the "behavior" layer:

- **HTML** = Structure (the skeleton)
- **CSS** = Presentation (the skin and clothes)
- **JavaScript** = Behavior (the muscles and brain)

Your First JavaScript Code

In `script.js`, type:

```
console.log("Hello, JavaScript!");
```

JS

Save the file and watch your browser automatically refresh. Check the console—you'll see your message!

What just happened?

- `console.log()` prints messages to the browser console
- Everything inside the parentheses and quotes is displayed
- The semicolon (`;`) marks the end of a statement (optional but good practice)

Part 2: Variables - Storing Information

Variables are containers that hold data. JavaScript has three ways to declare variables:

Experiment 1: Using `let`

Add this to your `script.js`:

```
// Using let - can be reassigned
let playerName = "Alex";
console.log("Player name:", playerName);

playerName = "Jordan"; // Changing the value
console.log("New player name:", playerName);
```

JS

Save and observe the console.

Key Points:

- `let` allows you to change the value later
- `//` creates a comment (ignored by JavaScript)
- We can reassign `playerName` to a different value

Experiment 2: Using `const`

Add below your previous code:

```
// Using const - cannot be reassigned
const PI = 3.14159;
console.log("Value of PI:", PI);

// Try to change it (this will cause an error!)
// PI = 3.14; // Uncomment this line to see the error
```

Key Points:

- `const` is for values that shouldn't change
- Use `const` by default, only use `let` when you need to reassign
- Constants are often written in UPPERCASE (convention for clarity)

Experiment 3: Variable Naming Rules**Add this code:**

```
// Valid variable names
let userName = "Sam";
let user_age = 25;
let $price = 99.99;
let _temp = 32;

// Invalid names (don't uncomment these!)
// let 2cool = "nope"; // Can't start with a number
// let user-name = "nope"; // Can't use hyphens
// let let = "nope"; // Can't use reserved words

console.log("All valid variables created!");
```

Rules:

- Start with a letter, `$`, or `_`
- Can contain letters, numbers, `$`, or `_`
- Cannot use reserved words (like `let`, `const`, `if`, etc.)
- Case-sensitive: `userName` and `username` are different!

Part 3: Data Types

JavaScript has several data types. Let's explore each one!

Type 1: Numbers

Add this code:

```
JS
console.log("\n--- NUMBERS ---");

let age = 25;
let price = 19.99;
let temperature = -5;
let infinity = Infinity;

console.log("Age:", age);
console.log("Price:", price);
console.log("Temperature:", temperature);
console.log("Infinity:", infinity);

// Basic math operations
let sum = 10 + 5;
let difference = 10 - 5;
let product = 10 * 5;
let quotient = 10 / 5;
let remainder = 10 % 3; // Modulo - gives remainder

console.log("10 + 5 =", sum);
console.log("10 - 5 =", difference);
console.log("10 * 5 =", product);
console.log("10 / 5 =", quotient);
console.log("10 % 3 =", remainder);
```

Key Points:

- JavaScript has only one number type (no separate integers and floats)
- Special values: **Infinity**, **-Infinity**, **NaN** (Not a Number)
- **\n** in strings creates a new line for better console readability

Type 2: Strings

Add this code:

```
console.log("\n--- STRINGS ---");

let firstName = "John";
let lastName = 'Doe'; // Single or double quotes both work
let greeting = `Hello, ${firstName}!`; // Template literal with backticks

console.log("First name:", firstName);
console.log("Last name:", lastName);
console.log(greeting);

// String operations
let fullName = firstName + " " + lastName; // Concatenation
console.log("Full name:", fullName);
console.log("Length:", fullName.length);
console.log("Uppercase:", fullName.toUpperCase());
console.log("Lowercase:", fullName.toLowerCase());

// Template literals (backticks)
let age2 = 30;
let message = `${firstName} is ${age2} years old`;
console.log(message);
```

Key Points:

- Strings are text data enclosed in quotes
- Three ways to create strings: `"double"`, `'single'`, ``backticks``
- Template literals (backticks) allow embedding variables with `${}`
- Strings have useful methods like `.length`, `.toUpperCase()`

Type 3: Booleans

Add this code:

```

console.log("\n--- BOOLEANS ---");

let isLoggedIn = true;
let hasPermission = false;

console.log("Logged in?", isLoggedIn);
console.log("Has permission?", hasPermission);

// Comparison operators return booleans
let age3 = 20;
console.log("Age ≥ 18?", age3 ≥ 18);
console.log("Age < 18?", age3 < 18);
console.log("Age === 20?", age3 === 20); // Strict equality
console.log("Age !== 25?", age3 !== 25); // Not equal

```

Key Points:

- Booleans have only two values: `true` or `false`
- Used for conditions and logic
- Comparison operators: `>`, `<`, `≥`, `≤`, `===`, `!==`
- Use `===` (strict equality) instead of `==` (avoid type coercion issues)

Type 4: Undefined and Null**Add this code:**

```

console.log("\n--- UNDEFINED & NULL ---");

let notAssigned; // Declared but not assigned
let emptyValue = null; // Intentionally empty

console.log("Not assigned:", notAssigned); // undefined
console.log("Empty value:", emptyValue); // null

// The difference
console.log("Type of undefined:", typeof notAssigned);
console.log("Type of null:", typeof emptyValue); // Returns "object" (quirk)

```

Key Points:

- `undefined` means "not yet assigned a value"
- `null` means "intentionally empty or no value"
- `undefined` happens automatically, `null` is assigned by you

Type 5: Arrays

Add this code:

```
JS
console.log("\n--- ARRAYS ---");

let fruits = ["apple", "banana", "orange"];
let numbers = [1, 2, 3, 4, 5];
let mixed = [1, "hello", true, null]; // Can mix types

console.log("Fruits:", fruits);
console.log("First fruit:", fruits[0]); // Arrays start at index 0
console.log("Second fruit:", fruits[1]);
console.log("Array length:", fruits.length);

// Array methods
fruits.push("grape"); // Add to end
console.log("After push:", fruits);

fruits.pop(); // Remove from end
console.log("After pop:", fruits);

fruits.unshift("mango"); // Add to beginning
console.log("After unshift:", fruits);
```

Key Points:

- Arrays store multiple values in order
- Access items with square brackets: `array[index]`
- Indexes start at 0
- Arrays are flexible and have many useful methods

Type 6: Objects

Add this code:


```
console.log("\n--- OBJECTS ---");

let person = {
  name: "Alice",
  age: 28,
  city: "New York",
  isStudent: false
};

console.log("Person object:", person);
console.log("Name:", person.name); // Dot notation
console.log("Age:", person["age"]); // Bracket notation

// Modifying objects
person.age = 29;
person.country = "USA"; // Adding new property
console.log("Updated person:", person);

// Nested objects
let employee = {
  name: "Bob",
  position: "Developer",
  contact: {
    email: "bob@example.com",
    phone: "555-1234"
  }
};

console.log("Employee:", employee);
console.log("Email:", employee.contact.email);
```

Key Points:

- Objects store key-value pairs
- Access properties with dot notation or bracket notation
- Objects are mutable (can be changed)
- Can contain any data types, including other objects

Part 4: Type Checking and Conversion

Add this code:

```
console.log("\n--- TYPE CHECKING ---");

// typeof operator
console.log("typeof 42:", typeof 42);
console.log("typeof 'hello':", typeof "hello");
console.log("typeof true:", typeof true);
console.log("typeof undefined:", typeof undefined);
console.log("typeof null:", typeof null); // Returns "object" (historical bug)
console.log("typeof []:", typeof []); // Returns "object"
console.log("typeof {}:", typeof {}); // Returns "object"

// Type conversion
console.log("\n--- TYPE CONVERSION ---");

let str = "42";
let num = Number(str); // String to number
console.log("String '42' to number:", num, typeof num);

let num2 = 42;
let str2 = String(num2); // Number to string
console.log("Number 42 to string:", str2, typeof str2);

let truthyValue = Boolean(1); // Truthy value
let falsyValue = Boolean(0); // Falsy value
console.log("Boolean(1):", truthyValue);
console.log("Boolean(0):", falsyValue);

// Falsy values in JavaScript
console.log("\nFalsy values:");
console.log("Boolean(0):", Boolean(0));
console.log("Boolean(''):", Boolean(""));
console.log("Boolean(null):", Boolean(null));
console.log("Boolean(undefined):", Boolean(undefined));
console.log("Boolean(NaN):", Boolean(NaN));
```

Part 5: Practice Exercises

Add this code to test your understanding:

```
console.log("\n--- PRACTICE EXERCISES ---");

// Exercise 1: Create a user profile
const userProfile = {
  username: "coder123",
  email: "coder@example.com",
  age: 22,
  isPremium: true,
  favoriteLanguages: ["JavaScript", "Python", "Go"]
};

console.log("User Profile:", userProfile);
console.log(`${userProfile.username} knows ${userProfile.favoriteLanguages.length} languages`);

// Exercise 2: Calculate total price
let itemPrice = 29.99;
let quantity = 3;
let taxRate = 0.08;

let subtotal = itemPrice * quantity;
let tax = subtotal * taxRate;
let total = subtotal + tax;

console.log(`\nOrder Summary:`);
console.log(`Item Price: ${itemPrice}`);
console.log(`Quantity: ${quantity}`);
console.log(`Subtotal: ${subtotal.toFixed(2)}`);
console.log(`Tax (8%): ${tax.toFixed(2)}`);
console.log(`Total: ${total.toFixed(2)}`);

// Exercise 3: String manipulation
let sentence = " JavaScript is AWESOME! ";
let cleaned = sentence.trim().toLowerCase();
let replaced = cleaned.replace("awesome", "amazing");

console.log("\nOriginal:", sentence);
console.log("Cleaned:", cleaned);
console.log("Replaced:", replaced);
```

Quick Reference Guide

JS

```
// VARIABLES
let variable = "can change";
const constant = "cannot change";

// DATA TYPES
let myNumber = 42;           // Number
let myString = "hello";     // String
let myBoolean = true;       // Boolean
let myUndefined;            // Undefined
let myNull = null;          // Null
let myArray = [1, 2, 3];     // Array
let myObject = { key: "value" }; // Object

// OPERATORS
// Arithmetic: + - * / % ** (exponent)
// Comparison: === !== > < ≥ ≤
// Logical: && (and) || (or) ! (not)

// USEFUL METHODS
// String: .length .toUpperCase() .toLowerCase() .trim() .replace()
// Array: .length .push() .pop() .shift() .unshift()
// Number: .toFixed() .toString()
```

Common Mistakes to Avoid

1. **Using `==` instead of `===`:** Always use strict equality (`===`)
2. **Forgetting to declare variables:** Always use `let` or `const`
3. **Case sensitivity:** `myVariable` ≠ `myvariable`
4. **Array indexing:** Remember arrays start at 0, not 1
5. **String quotes:** Make sure opening and closing quotes match

Next Steps

Now that you understand variables and data types, you're ready to learn:

1. **Operators and Expressions** - More complex calculations
2. **Control Flow** - if/else statements and switch cases
3. **Loops** - for, while, and iteration
4. **Functions** - Reusable code blocks
5. **DOM Manipulation** - Making your webpage interactive!

Challenge: Try creating your own examples for each data type. Modify the code and experiment. Breaking things is the best way to learn!

Pro Tip: Keep this file and refer back to it. Add your own notes as comments in the code. The more you experiment, the better you'll understand JavaScript!